# BITÁCORA DE PRÁCTICA 10: POLIMORFISMO

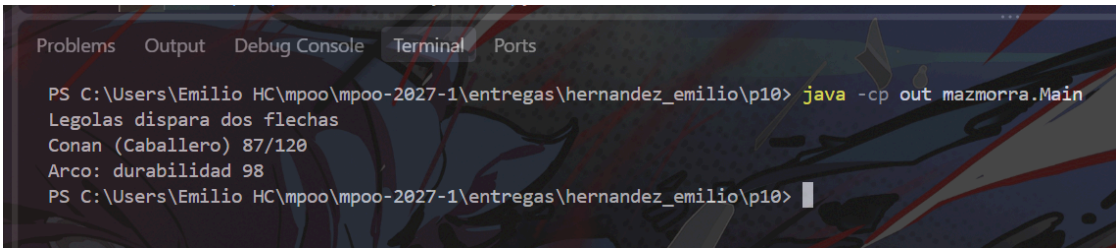
## Datos Generales del Equipo

| Integrante | Número de Cuenta |
| --- | --- |
| Hernandez Crisostomo Emilio | 323165686 |
| Esquivel Cortes Iker Alonso | 323004488 |

---

# 1. CAPTURAS DE PANTALLA

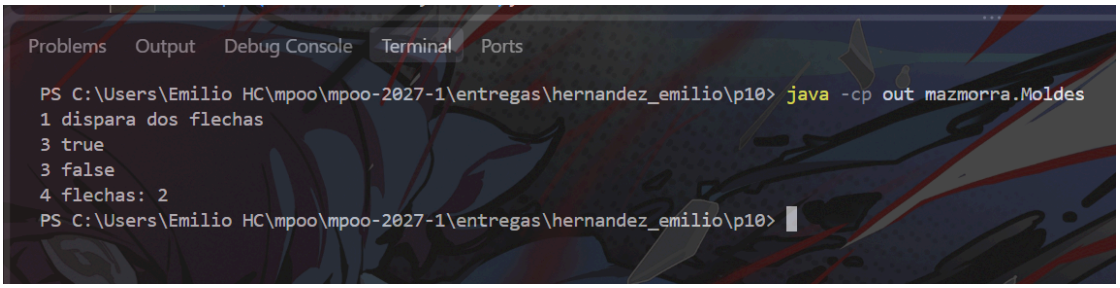
## Ejecución de Main.java




## Ejecución de Comparar.java



### Ejecución de Moldes.java




---

## 2. RESPUESTAS AL CUESTIONARIO TEÓRICO

### Pregunta 1: Una orden, muchas respuestas

En `Personaje.java`, la línea clave que llama al ataque es:

```
int base = herramienta.golpear();
```

Cuando Legolas le quita 33 de vida a Conan, la regla de golpear (las dos flechas, que dan un total de 30 de daño base) la decide la clase del objeto real en memoria (`Arco`) gracias al polimorfismo y al enlace dinámico. El +3 extra lo decide el propio personaje (Arquero) al aplicar su método calcularDanio(base), sumándole 3 a los 30 base. Todo esto lo resuelve Java mirando el objeto que está ejecutándose en ese momento, no el tipo con el que se declaró la variable.

### Pregunta 2: Sin y con polimorfismo

Si mañana entra una clase nueva como `Hacha` al proyecto:

**En la mitad sin polimorfismo (`sinPolimorfismo`):** tendríamos que abrir manualmente el archivo `Comparar.java` y meter un bloque nuevo con `else if (h instanceof Hacha)` para indicarle qué grito dar y cómo calcular su golpe. Si se nos olvida ponerlo, el código cae directamente en el `else` general e imprime *"no se que es esto"*.

**En la mitad con polimorfismo (`conPolimorfismo`):** no hay que moverle absolutamente nada a `Comparar.java`. El ciclo simplemente llama a `h.grito()` y `h.golpear()`, y la clase `Hacha` solita ya sabe qué responder.

### Pregunta 3: Los moldes (Explicación de errores en línea 2 y línea 5)

**Línea 2 descomentada:** `h.getFlechas();`

- **Error:** `cannot find symbol: method getFlechas() location: variable h of type Herramienta`
- **Explicación:** Este problema lo detecta el compilador (`javac`) antes de correr el programa. La variable `h` fue declarada como tipo `Herramienta`, y la clase `Herramienta` no tiene ningún método llamado `getFlechas()`. Aunque por dentro sepamos que el objeto guardado es un `Arco`, el compilador solo revisa lo que la variable le permite ver.

**Línea 5 descomentada:** `Espada c = (Espada) h;`

- **Error:** `java.lang.ClassCastException: class mazmorra.Arco cannot be cast to class mazmorra.Espada`
- **Explicación:** Esta línea pasa la prueba del compilador porque sintácticamente hacer un cast entre clases de la misma jerarquía es válido. Sin embargo, revienta en tiempo de ejecución (Java al correr). Cuando la máquina virtual evalúa la conversión, revisa el objeto real y se da cuenta de que es un `Arco`, por lo que resulta imposible forzarlo a convertirse en una `Espada`.